

Flujo de trabajo para LaTeX en VS Code

Guía práctica para investigación académica y colaboración con Overleaf

Diseñado para: Kristian López Vargas Entorno principal: macOS, Visual Studio Code, Git y Overleaf opcional
Actualización: agosto de 2026

Cómo usar esta guía

Esta guía está pensada para retomarse después de meses sin tocar LaTeX. Las secciones 1 a 8 cubren el trabajo local. Las secciones 9 a 11 explican colaboración. La sección 12 sirve como prueba de humo y la 13 como referencia rápida.

- **RECOMENDACIÓN:** opción por defecto salvo que exista un motivo explícito para cambiarla.
- **CUIDADO:** riesgo real de pérdida de trabajo, historial o tiempo.
- **COMPROBACIÓN:** gesto o comando que demuestra que una parte funciona.

Ruta según lo que necesitas

Si quieres...	Ve directamente a...
Instalar y comprobar el entorno	Sección 3
Activar el salto entre código y PDF	Secciones 2 y 5
Recuperar un documento que no compila	Sección 7
Trabajar con coautores en Overleaf	Secciones 9 a 11
Confirmar que toda la instalación funciona	Sección 12

Los nombres de menús y algunas preferencias pueden cambiar entre versiones de VS Code o LaTeX Workshop. Los comandos, límites y recomendaciones de integración se contrastaron con documentación oficial consultada el 3 de agosto de 2026; las fuentes aparecen al final.

1. La arquitectura recomendada

La mayoría de los problemas del LaTeX colaborativo no son problemas de LaTeX. Son problemas de autoridad ambigua: nadie sabe cuál copia manda.

Función	Herramienta	Regla operativa
Edición y compilación	VS Code + MacTeX/TeX Live	Aquí se escribe, compila y depura.
Historial y revisión	Git + GitHub	Aquí vive la historia canónica y se revisan cambios.
Colaboración en navegador	Overleaf, opcional	Se usa cuando un coautor no quiere instalar un entorno local.

RECOMENDACIÓN: edita localmente en VS Code, conserva el historial en GitHub y usa Overleaf como superficie de colaboración solo cuando el equipo lo necesite.

CUIDADO. La sincronización directa Overleaf-GitHub es manual, no automática.[6] Si alguien escribe en Overleaf mientras otra persona modifica la misma línea en VS Code, el conflicto no se resuelve solo. El modo híbrido requiere el protocolo de turnos de la sección 11.

Tres modos viables

Modo	Ventaja	Riesgo	Cuándo elegirlo
Local + GitHub	Historial completo, ramas y PR	Todos necesitan Git	Trabajo individual o equipo técnico
Overleaf primero	Colaboración inmediata en navegador	Historial menos granular	Equipo no técnico, documento corto
Híbrido	Combina edición local y navegador	Divergencia entre copias	Autor principal técnico y coautores por turnos

Empieza con local + GitHub. Pasa al híbrido cuando un coautor concreto lo requiera y el equipo acepte la coordinación adicional. Overleaf Git y GitHub Sync son funciones premium; el número de colaboradores también depende del plan.[4-6] Comprueba la cobertura de tu cuenta, grupo o suscripción institucional antes de diseñar el flujo. Cada colaborador que acceda por Git debe usar su propia cuenta y su propio token.[5]

2. El nombre correcto: SyncTeX

La frase informal "click to seek" o "click to text" describe SyncTeX, el mapa que conecta una posición del código fuente con una posición del PDF. Al compilar, TeX crea normalmente `main.synctex.gz`.

Del código al PDF: forward search

Coloca el cursor en una línea del `.tex` y ejecuta SyncTeX from cursor:

- macOS: `Cmd+Option+J`.
- Windows/Linux: `Ctrl+Alt+J`.

Del PDF al código: inverse search

En el visor PDF interno de LaTeX Workshop, esta guía configura un gesto único y fácil de recordar:

- doble clic sobre el PDF en macOS, Windows o Linux.

La opción predeterminada de la extensión se llama `ctrl-click`, pero la traducción de modificadores en macOS puede resultar confusa y `Ctrl+click` suele abrir el menú contextual. Por eso la configuración de la sección 5 fija explícitamente `double-click`.

RECOMENDACIÓN: usa el visor PDF interno, en una pestaña de VS Code. Es la ruta más estable y requiere menos configuración.

La integración SyncTeX con visores externos no está oficialmente soportada por LaTeX Workshop; su documentación además conserva una advertencia de funcionamiento para esa ruta.^[1] Skim en macOS, SumatraPDF en Windows y otros visores son alternativas avanzadas, no el punto de partida.

Diagnóstico rápido de SyncTeX

- 1 Confirma que existe `out/main.synctex.gz` o `main.synctex.gz`.
- 2 Confirma que la compilación usó `-synctex=1`.
- 3 Abre el PDF en el visor interno de LaTeX Workshop.
- 4 Verifica que la extensión detectó el archivo raíz correcto.
- 5 Recompila si el PDF o el mapa quedaron antiguos.

Los saltos pueden ser aproximados dentro de tablas, fórmulas o macros complejas. Aterrizar unas líneas antes o después no necesariamente indica un fallo.

3. Instalación mínima

La extensión de VS Code no instala LaTeX. Se necesitan tres piezas separadas:

- 1 Visual Studio Code, el editor.
- 2 Una distribución LaTeX, que aporta `latexmk`, `pdflatex`, `SyncTeX` y paquetes.
- 3 La extensión LaTeX Workshop, que conecta compilación, visor, navegación y diagnósticos.[3]

macOS: MacTeX o BasicTeX

Opción	Contenido	Tamaño aproximado	Recomendación
MacTeX 2026	TeX Live completo	Más de 6 GB	Mejor para trabajo académico estable
BasicTeX 2026	Subconjunto mínimo	Alrededor de 140 MB	Útil si el disco o la red son limitantes

TUG recomienda MacTeX completo para la mayoría de usuarios porque evita buscar paquetes faltantes.[2] BasicTeX funciona, pero puede requerir instalaciones adicionales con `tlmgr`.

En Windows, TeX Live o MiKTeX son opciones habituales. En Linux, puede usarse TeX Live. La receta predeterminada de LaTeX Workshop usa `latexmk`, que requiere Perl.[1]

Pasos

- 1 Instala VS Code.
- 2 Instala MacTeX desde <https://tug.org/mactex/>.
- 3 Cierra y vuelve a abrir la terminal y VS Code.
- 4 En Extensions, instala LaTeX Workshop, publicada por James Yu.
- 5 En macOS, habilita `code` desde la Command Palette con Shell Command: Install 'code' command in PATH.

COMPROBACIÓN en terminal

```
code --version
latexmk --version
pdflatex --version
synctex help
git --version
perl --version
which latexmk pdflatex biber bibtex
```

Cada comando debe devolver una ruta o versión. Si la terminal encuentra LaTeX y VS Code no, cierra VS Code por completo y ábrelo desde la terminal con `code ..`

4. Estructura del proyecto

```

mi-articulo/
├── main.tex
├── references.bib
├── sections/
│   ├── 01-introduccion.tex
│   ├── 02-datos.tex
│   ├── 03-resultados.tex
│   └── 04-conclusion.tex
├── figures/
├── .vscode/settings.json
├── .gitignore
└── README.md

```

Reglas simples:

- Un solo archivo raíz: `main.tex`.
- Una sección por archivo para producir diffs claros y reducir conflictos.
- Figuras vectoriales en PDF cuando sea posible; PNG para capturas.
- Una bibliografía: `references.bib`.
- Rutas y nombres de archivo cortos, estables y sin caracteres especiales.

`main.tex` **mínimo**

```

\documentclass[11pt,a4paper]{article}
\usepackage[T1]{fontenc}
\usepackage[spanish,es-tabla]{babel}
\usepackage{csquotes}
\usepackage{microtype,graphicx,booktabs}
\usepackage[hidelinks]{hyperref}
\usepackage[backend=biber,style=apa,natbib=true]{biblatex}
\addbibresource{references.bib}
\graphicspath{{figures/}}

\title{Título provisional del artículo}
\author{Nombre Apellido}
\date{\today}

\begin{document}
\maketitle
\begin{abstract}
Resumen de 150 palabras.
\end{abstract}
\input{sections/01-introduccion}
\input{sections/02-datos}
\input{sections/03-resultados}
\input{sections/04-conclusion}
\printbibliography
\end{document}

```

Al comienzo de cada archivo dentro de `sections/`, indica el raíz con una ruta relativa:

```

% !TEX root = ../main.tex
\section{Introducción}
\label{sec:intro}
Este párrafo prueba la compilación y SyncTeX.

```

La magic comment ayuda a LaTeX Workshop cuando editas un archivo incluido. En `main.tex` no es necesaria.

5. Configuración recomendada de VS Code

LaTeX Workshop usa `latexmk` por defecto. Su receta incluye `-synctex=1`, `-interaction=nonstopmode`, `-file-line-error`, `-pdf` y un directorio de salida.[1] No hace falta copiar una receta personalizada para empezar.

Crea `.vscode/settings.json`:

```
{
  "latex-workshop.latex.autoBuild.run": "onSave",
  "latex-workshop.latex.outDir": "%DIR%/out",
  "latex-workshop.view.pdf.viewer": "tab",
  "latex-workshop.synctex.afterBuild.enabled": true,
  "latex-workshop.view.pdf.internal.synctex.keybinding": "double-click",
  "latex-workshop.latex.autoClean.run": "never",
  "latex-workshop.latex.clean.fileTypes": [
    "*.aux", "*.bbl", "*.blg", "*.idx", "*.ind",
    "*.lof", "*.lot", "*.out", "*.toc", "*.fls",
    "*.fdb_latexmk", "*.log", "*.bcf", "*.run.xml"
  ],
  "editor.wordWrap": "on",
  "[latex]": {
    "editor.formatOnSave": false,
    "editor.rulers": [90]
  }
}
```

Decisiones importantes:

- Compilar al guardar es predecible y no recompila con cada tecla.
- `out/` mantiene limpia la raíz del proyecto.
- La limpieza automática queda en `never` para conservar el `.log` y el `.blg` cuando algo falla; se limpia manualmente después del diagnóstico.
- La lista de limpieza no incluye `*.synctex.gz` ni el PDF.
- Formatear manualmente evita commits enormes que ocultan cambios sustantivos.
- `latex-workshop.synctex.afterBuild.enabled` puede cambiarse a `false` si no quieres saltar automáticamente después de cada build.

La línea `double-click` reemplaza deliberadamente el gesto predeterminado. Después de cambiarla, reinicia VS Code o vuelve a abrir el visor PDF para asegurar que la preferencia se aplique.[1]

6. El ciclo diario

- 1 Abre la carpeta completa: `code ~/proyectos/mi-articulo`.
- 2 Actualiza antes de editar: `git pull --rebase origin main`.
- 3 Escribe en `sections/`; una frase por línea produce diffs más legibles.
- 4 Guarda con `Cmd+S` o `Ctrl+S` para compilar.
- 5 Revisa Problems y, si hace falta, el log de LaTeX Workshop.
- 6 Usa `Cmd+Option+J` o `Ctrl+Alt+J` para ir al PDF.
- 7 Usa doble clic en el PDF para regresar al código.
- 8 Confirma un cambio pequeño y coherente.

```
git add sections/03-resultados.tex references.bib
git commit -m "Revisa interpretación del modelo 2 y añade cita"
git push origin main
```

RECOMENDACIÓN: un commit debe poder describirse en una sola frase. Evita mensajes como "cambios varios".

Qué revisar antes de un commit

- El documento compila desde cero.
- No hay errores nuevos en Problems.
- Las citas y referencias se resolvieron.
- Las figuras necesarias están versionadas.
- `git diff --check` no reporta errores de espacios.
- `git status` no incluye datos confidenciales ni auxiliares.

7. Depuración de compilación

Sigue este orden; resuelve más errores por minuto que cambiar opciones al azar.

- 1 Archivo raíz. Confirma que compilas `main.tex`, no un fragmento.
- 2 PATH. Ejecuta `which latexmk pdflatex`; si solo funciona fuera de VS Code, reabre con `code ..`
- 3 Compilación manual. El error crudo puede ser más claro:

```
latexmk -pdf -synctex=1 -interaction=nonstopmode \
  -file-line-error -outdir=out main.tex
```

- 1 Primer error real. Corrige la primera línea importante del log, no los veinte errores derivados.
- 2 Paquete ausente. `File 'xxx.sty' not found` indica un paquete faltante. Con BasicTeX es esperable.
- 3 Auxiliares antiguos. Limpia el directorio de salida y recompila:

```
latexmk -C -outdir=out main.tex
latexmk -pdf -synctex=1 -outdir=out main.tex
```

`latexmk -C` borra auxiliares y también el PDF del directorio de salida; la siguiente línea lo reconstruye.

- 1 Bibliografía. `biblatex` suele usar `biber`; `natbib` clásico suele usar `bibtex`. Examina el `.bbl` y las claves de `references.bib`.
- 2 Bisección. Comenta temporalmente bloques `\input{}` hasta aislar la sección que rompe el build.

CUIDADO. Un aviso `Overfull \hbox` suele ser tipográfico; un error que detiene el build no lo es. No mezcles ambos niveles de severidad.

8. Higiene de Git

Versionar

- .tex, .bib, .cls, .sty y configuración del proyecto.
- Figuras finales incluidas en el manuscrito.
- Scripts y datos reproducibles cuando la licencia, tamaño y privacidad lo permitan.
- README.md, .gitignore y .vscode/settings.json.

No versionar

- Auxiliares: .aux, .log, .fls, .fdb_latexmk, .synctex.gz, etc.
- out/.
- Datos personales o confidenciales.
- El PDF de cada compilación diaria.

```
# Salida y auxiliares
out/
*.pdf
*.synctex.gz
*.aux
*.log
*.toc
*.lof
*.lot
*.out
*.fls
*.fdb_latexmk
*.bbl
*.blg
*.bcf
*.run.xml

# Sistema
.DS_Store
Thumbs.db

# Excepciones que sí se versionan
!figures/*.pdf
!releases/*.pdf
```

¿Versionar el PDF final?

Situación	Decisión recomendada
Desarrollo diario	No; es binario y crece sin aportar diffs útiles.
Envío, preprint o versión citada	Sí, preferiblemente como release etiquetada.
Coautores que no compilan	Solo en hitos, dentro de releases/.

Para arXiv y algunos flujos editoriales puede ser necesario entregar el .bbl. Aunque se ignore durante el desarrollo, inclúyelo explícitamente en el paquete de envío o añádelo al hito con `git add -f out/main.bbl`.

Comprueba que las figuras PDF no quedaron ignoradas:

```
git status --ignored
git check-ignore -v figures/fig-modelo.pdf
```

9. Qué ofrece Overleaf

Overleaf permite edición simultánea en navegador, comentarios, control de cambios y roles de edición, revisión o lectura.[4] Es excelente cuando la principal barrera es instalar herramientas, pero no reemplaza automáticamente el historial de Git.

Opciones

Opción	Cómo funciona	Principal límite
Colaboración nativa	Todos trabajan en Overleaf	El número de colaboradores depende del plan
Git de Overleaf	Overleaf funciona como remoto Git	Premium; una historia lineal master
GitHub Sync	Push/pull manual desde la interfaz	Premium; sin ramas ni Git LFS
Dropbox Sync	Sincronización bidireccional de archivos	Premium; no es control de versiones[8]

La sincronización directa con GitHub funciona solo con `github.com`, no con GitHub Enterprise. No puede enlazar un proyecto Overleaf existente con un repositorio GitHub existente; hay que crear uno desde el otro. Los conflictos producen una rama adicional en el repositorio GitHub, con la versión de Overleaf, que luego se fusiona localmente o mediante un pull request.[6]

Tampoco admite Git LFS ni submódulos internos. Overleaf recomienda menos de 100 archivos por commit y menos de 100 MB por proyecto para esta integración.[6]

CUIDADO. Los pulls desde GitHub pueden desplazar comentarios y cambios rastreados en Overleaf. Resuelve o registra esas conversaciones antes de sincronizar.[6]

10. Overleaf como remoto Git

La integración Git de Overleaf permite clonar el proyecto y tratarlo como un remoto. Es premium. La autenticación actual usa el usuario literal `git` y un token personal como contraseña.[5]

```
git clone https://git.overleaf.com/<ID_DEL_PROYECTO>
cd <CARPETA_CLONADA>
# Username: git
# Password: token personal de Overleaf
```

Nunca incrustes el token en la URL, el repositorio, un chat o una captura. Usa un gestor de credenciales; en macOS:

```
git config --global credential.helper osxkeychain
```

Overleaf Git acepta una sola historia lineal cuyo nombre remoto es `master`.[7] La rutina básica es:

```
git pull --rebase
# editar, compilar y confirmar
git push
```

Dos remotos: GitHub y Overleaf

Esta secuencia asume un repositorio GitHub nuevo y vacío. Si GitHub y Overleaf ya tienen historias distintas, usa el procedimiento de reconciliación avanzada de la fuente 7.

```
git remote rename origin overleaf
git remote add origin https://github.com/<usuario>/<repo>.git
git branch -m master main
git push -u origin main
git remote -v
```

Si tu rama local es `main`, el ciclo explícito es:

```
git pull --rebase overleaf master
git pull --rebase origin main
latexmk -pdf -synctex=1 -outdir=out main.tex
git push origin main
git push overleaf main:master
```

El mapeo `main:master` importa porque el lado Overleaf solo acepta su rama `master`.[7] Este patrón de dos remotos también permite usar GitLab o GitHub Enterprise como autoridad principal, aunque no exista sincronización directa desde Overleaf.[7]

11. Protocolo de turnos para el modo híbrido

Regla base: en cada momento existe una única sede autorizada para editar: VS Code/GitHub u Overleaf, nunca ambas.

- 1 Acordar la sede. Anuncia: "Paso la autoridad a Overleaf hasta el viernes".
- 2 Cerrar el turno. Compila, verifica, hace commit y push.
- 3 Sincronizar al destino. Empuja a Overleaf o usa GitHub Sync manual.
- 4 Verificar allí. Abre Overleaf y confirma que compila.
- 5 Anunciar el traspaso. Incluye el commit: "Overleaf está al día en a1b2c3d".
- 6 Actualizar antes de escribir. Quien recibe hace pull/sync antes de tocar archivos.
- 7 Devolver el turno. Repite los pasos en sentido inverso.

Antes de llevar GitHub hacia Overleaf

- Resuelve o documenta comentarios y cambios rastreados pendientes.
- Confirma que el repositorio local está limpio.
- Fusiona cualquier PR pendiente.
- Compila localmente.
- Sincroniza y vuelve a compilar en Overleaf.

Si aparece un conflicto

Detén las ediciones. En GitHub Sync, Overleaf empuja una rama nueva al repositorio GitHub.[6] Revísala, crea o usa el PR, resuelve el conflicto, compila y solo después devuelve la autoridad al equipo.

Los turnos de días son más seguros que los de horas. Sincronizar constantemente multiplica las oportunidades de conflicto.

12. Prueba de humo de 10 minutos

Minutos 0-2: entorno

```
code --version
latexmk --version
pdflatex --version
synctex help
git --version
```

Minutos 2-4: proyecto

Crea una carpeta con `main.tex` y `sections/01-introduccion.tex`. Para aislar el entorno de la bibliografía, usa este raíz mínimo:

```
\documentclass[11pt,a4paper]{article}
\usepackage[T1]{fontenc}
\usepackage[spanish]{babel}
\usepackage[hidelinks]{hyperref}
\begin{document}
\input{sections/01-introduccion}
\end{document}
```

Minutos 4-6: línea de comandos

```
latexmk -pdf -synctex=1 -interaction=nonstopmode \
  -file-line-error -outdir=out main.tex
ls -la out/main.pdf out/main.synctex.gz
```

Ambos archivos deben existir. Si no existen, el problema está en la distribución LaTeX, no en el visor de VS Code.

Minutos 6-8: VS Code

Ejecuta `code .`, abre `main.tex` y guarda. La primera vez, abre la Command Palette y ejecuta `View LaTeX PDF file in VS Code tab`; confirma que la pestaña aparece y no hay errores en Problems.

Minutos 8-9: forward search

Pon el cursor en la introducción y pulsa `Cmd+Option+J` en macOS o `Ctrl+Alt+J` en Windows/Linux.

Minutos 9-10: inverse search

Haz doble clic sobre el párrafo en el visor PDF interno. El editor debe saltar a `sections/01-introduccion.tex`.

Si la prueba falla, el minuto exacto indica qué sección revisar: instalación, compilación o SyncTeX.

13. Hoja de referencia rápida

Acción	macOS	Windows/Linux
Paleta de comandos	Cmd+Shift+P	Ctrl+Shift+P
Guardar y compilar	Cmd+S	Ctrl+S
Fuente al PDF	Cmd+Option+J	Ctrl+Alt+J
PDF a fuente	Doble clic	Doble clic
Abrir carpeta	code .	code .

Comandos esenciales

```
# Compilar con SyncTeX
latexmk -pdf -synctex=1 -interaction=nonstopmode \
  -file-line-error -outdir=out main.tex

# Limpiar y reconstruir
latexmk -C -outdir=out main.tex
latexmk -pdf -synctex=1 -outdir=out main.tex

# Git diario
git pull --rebase origin main
git add <archivos>
git commit -m "Cambio coherente y descriptivo"
git push origin main

# Dos remotos
git pull --rebase overleaf master
git push origin main
git push overleaf main:master
```

Siete recordatorios

- 1 Abre la carpeta, no el archivo suelto.
- 2 Usa % !TEX root = ../main.tex en los archivos dentro de sections/.
- 3 Sin -synctex=1 no hay navegación bidireccional.
- 4 No borres *.synctex.gz automáticamente.
- 5 Corrige el primer error real del log.
- 6 Decide si el PDF final se versiona; no lo hagas por accidente.
- 7 En modo híbrido, una sola sede edita a la vez.

Fuentes oficiales

Consultadas el 3 de agosto de 2026.

1. LaTeX Workshop, instalación, compilación y visualización. Requisitos, receta `latexmk`, SyncTeX y visores.
<https://github.com/James-Yu/LaTeX-Workshop/wiki/Install> <https://github.com/James-Yu/LaTeX-Workshop/wiki/Compile>
<https://github.com/James-Yu/LaTeX-Workshop/wiki/View>
2. TeX Users Group, MacTeX. MacTeX 2026, BasicTeX y recomendaciones. <https://tug.org/mactex/> <https://tug.org/mactex/morepackages.html>
3. LaTeX Workshop, repositorio oficial. Funciones de compilación, visor, autocompletado y diagnósticos.
<https://github.com/James-Yu/LaTeX-Workshop>
4. Overleaf, colaboración y permisos. Edición simultánea, revisión, comentarios y límites de colaboradores.
<https://docs.overleaf.com/collaborating/collaborating-in-overleaf> <https://docs.overleaf.com/collaborating/sharing-a-project>
5. Overleaf, integración Git y tokens. Acceso como remoto Git y autenticación personal.
<https://docs.overleaf.com/integrations-and-add-ons/git-integration-and-github-synchronization/git>
<https://docs.overleaf.com/integrations-and-add-ons/git-integration-and-github-synchronization/git/git-integration-authentication-tokens>
6. Overleaf, GitHub synchronization. Sincronización manual, conflictos y limitaciones.
<https://docs.overleaf.com/integrations-and-add-ons/git-integration-and-github-synchronization/github-synchronization>
7. Overleaf, operaciones Git avanzadas. Rama `master` y patrón de dos remotos.
<https://docs.overleaf.com/integrations-and-add-ons/git-integration-and-github-synchronization/git-integration/advanced-git-operations>
8. Overleaf, Dropbox. Sincronización bidireccional y precauciones. <https://docs.overleaf.com/integrations-and-add-ons/dropbox>

Conclusión: el mejor flujo no es una aplicación única. Es una separación clara entre edición local, historial verificable y colaboración por turnos.